



Python Programming Fundamentals

Parameters and Return Values

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. Positional Parameters
2. Keyword Arguments
3. Default Parameter Values
4. The return Statement
5. Multiple Return Values
6. Designing Good Functions



Outline

1. Positional Parameters

2. Keyword Arguments

3. Default Parameter Values

4. The return Statement

5. Multiple Return Values

6. Designing Good Functions



1. Positional Parameters

Parameters are variables that receive values when the function is called. **Arguments** are the values passed by the caller.

```
def greet(name, time_of_day):      # name, time_of_day are parameters
    print(f"Good {time_of_day}, {name}!")
```

```
greet("Alice", "morning")          # positional – order matters
greet("Bob", "evening")
```



1. Positional Parameters



1. Positional Parameters

Parameters are variables that receive values when the function is called. **Arguments** are the values passed by the caller.

```
def greet(name, time_of_day):      # name, time_of_day are parameters
    print(f"Good {time_of_day}, {name}!")
```

```
greet("Alice", "morning")          # positional – order matters
greet("Bob", "evening")
```

Positional arguments are matched to parameters **in order** — the first argument goes to the first parameter, the second to the second, and so on.

```
def power(base, exponent):
    return base ** exponent
```



1. Positional Parameters

```
print(power(2, 8))    # 2^8 = 256
print(power(8, 2))    # 8^2 = 64 (different order, different result!)
```



Outline

1. Positional Parameters
- 2. Keyword Arguments**
3. Default Parameter Values
4. The return Statement
5. Multiple Return Values
6. Designing Good Functions



2. Keyword Arguments

Keyword arguments are passed by **name**, so order doesn't matter.

```
def create_user(username, email, age):  
    print(f"User: {username}, Email: {email}, Age: {age}")  
  
# Keyword arguments – any order  
create_user(email="alice@example.com", age=25, username="alice99")
```



2. Keyword Arguments

Keyword arguments are passed by **name**, so order doesn't matter.

```
def create_user(username, email, age):  
    print(f"User: {username}, Email: {email}, Age: {age}")
```

Keyword arguments – any order

```
create_user(email="alice@example.com", age=25, username="alice99")
```

You can **mix** positional and keyword arguments – positional must come first:

```
create_user("alice99", age=25, email="alice@example.com") # OK  
# create_user(username="alice99", "alice@example.com", 25) # SyntaxError
```



Outline

1. Positional Parameters
2. Keyword Arguments
- 3. Default Parameter Values**
4. The return Statement
5. Multiple Return Values
6. Designing Good Functions



3. Default Parameter Values

Give a parameter a **default value** so it becomes optional at the call site.

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")
```

```
greet("Alice")           # Hello, Alice!
greet("Bob", "Hi")       # Hi, Bob!
greet("Carol", greeting="Good morning") # Good morning, Carol!
```



3. Default Parameter Values



3. Default Parameter Values

Give a parameter a **default value** so it becomes optional at the call site.

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")
```

```
greet("Alice")           # Hello, Alice!
greet("Bob", "Hi")       # Hi, Bob!
greet("Carol", greeting="Good morning") # Good morning, Carol!
```

Practical example — configurable repeat:

```
def repeat_print(message, times=3, separator="-"):
    for i in range(times):
        print(message)
        print(separator * len(message))
```

```
repeat_print("Hello")    # 3 times, dashes
```



3. Default Parameter Values

```
repeat_print("Warning!", times=5)    # 5 times, dashes  
repeat_print("Hi", times=2, separator=="")    # 2 times, equals
```



Outline

1. Positional Parameters
2. Keyword Arguments
3. Default Parameter Values
- 4. The return Statement**
5. Multiple Return Values
6. Designing Good Functions



4. The return Statement

return sends a value back to the caller and **immediately exits** the function.

```
def square(n):  
    return n * n
```

```
result = square(7)          # result = 49  
print(square(5) + 1)        # 26  
print(square(square(3)))    # 81
```



4. The return Statement



4. The return Statement

return sends a value back to the caller and **immediately exits** the function.

```
def square(n):  
    return n * n
```

```
result = square(7)          # result = 49  
print(square(5) + 1)        # 26  
print(square(square(3)))    # 81
```

A function can have **multiple return statements** — the first one reached exits the function:

```
def absolute_value(n):  
    if n < 0:  
        return -n          # exits here for negatives  
    return n                # exits here for non-negatives
```



4. The return Statement

```
print(absolute_value(-5))    # 5  
print(absolute_value(7))     # 7
```



Outline

1. Positional Parameters
2. Keyword Arguments
3. Default Parameter Values
4. The return Statement
- 5. Multiple Return Values**
6. Designing Good Functions



5. Multiple Return Values

Python functions can return **multiple values** as a tuple.

```
def min_max(numbers):  
    """Return the minimum and maximum of a list."""  
    return min(numbers), max(numbers)  
  
lo, hi = min_max([3, 1, 7, 4, 2, 8, 5])  
print(f"Min: {lo}, Max: {hi}")    # Min: 1, Max: 8
```



5. Multiple Return Values



5. Multiple Return Values

Python functions can return **multiple values** as a tuple.

```
def min_max(numbers):  
    """Return the minimum and maximum of a list."""  
    return min(numbers), max(numbers)
```

```
lo, hi = min_max([3, 1, 7, 4, 2, 8, 5])  
print(f"Min: {lo}, Max: {hi}")    # Min: 1, Max: 8
```

Tuple unpacking on the left side:

```
def divide_with_remainder(a, b):  
    quotient = a // b  
    remainder = a % b  
    return quotient, remainder
```

```
q, r = divide_with_remainder(17, 5)
```




5. Multiple Return Values

```
print(f"17 ÷ 5 = {q} remainder {r}")    # 17 ÷ 5 = 3 remainder 2
```

```
# Or get the full tuple:
```

```
result = divide_with_remainder(100, 7)  
print(result)    # (14, 2)
```



Outline

1. Positional Parameters
2. Keyword Arguments
3. Default Parameter Values
4. The return Statement
5. Multiple Return Values
- 6. Designing Good Functions**



6. Designing Good Functions

Good functions:

- Do **one thing** well (single responsibility)
- Have a clear, descriptive name
- Have a docstring
- Accept inputs as parameters
- Return output via return (don't print inside unless displaying is the function's job)



6. Designing Good Functions



6. Designing Good Functions

Good functions:

- Do **one thing** well (single responsibility)
- Have a clear, descriptive name
- Have a docstring
- Accept inputs as parameters
- Return output via return (don't print inside unless displaying is the function's job)

BAD – function does too many things and mixes I/O with logic

```
def process():  
    name = input("Name: ")  
    score = float(input("Score: "))  
    grade = "Pass" if score >= 60 else "Fail"  
    print(f"{name}: {grade}")
```

GOOD – separate concerns



6. Designing Good Functions

```
def get_input(prompt):  
    return input(prompt)  
  
def calculate_grade(score):  
    return "Pass" if score >= 60 else "Fail"  
  
def display_result(name, grade):  
    print(f"{name}: {grade}")  
  
name = get_input("Name: ")  
score = float(get_input("Score: "))  
grade = calculate_grade(score)  
display_result(name, grade)
```



Thanks for Watching - Any questions?

